Theoretical Computer Science

FINITE AUTOMATA

Vietnamese-German University

Lecturers: Ngoc Tran, Ph.D.

Viet To, Ph.D.

Content

- Deterministic Finite Accepters (DFA).
- Nondeterministic Finite Accepters (NFA).
- Equivalence between DFA vs NFA.
- Reduction of the Number of States in Finite Automata.

Deterministic Finite Accepters (DFA)

- Definition 2.1: A deterministic finite accepter (dfa) is defined by the quintuple:

$$M = (Q, \Sigma, \delta, q_0, F)$$

Q is a finite set of internal states,

Σ is a finite set of symbols called the input alphabet,

$\delta: Q \times \Sigma \rightarrow Q$ is a transition function,

$q_0 \in Q$ is the initial state (q-nought),

$F \subseteq Q$ is a set of final states.

Deterministic Finite Accepters (DFA)

DFA Operation

- At the initial time, it is assumed to be in the initial state q_0, with its input mechanism on the leftmost symbol of the input string.
- The input mechanism can move only from left to right of the input string and reads exactly one symbol on each step.
- During each move of the automaton, the input mechanism advances one position to the right, so each move consumes one input symbol.

Deterministic Finite Accepters (DFA)

DFA Operation (cont.)

- When the end of the string is reached, the string is <u>accepted</u> if the automaton is in one of its final states. Otherwise, the string is <u>rejected</u>.
- The transitions from one internal state to another are governed by the transition function δ.
  - For example, if $\delta(q_0, a) = q_1$, then if the dfa is in state q_0 and the current input symbol is a, the dfa will go into state q_1.

Deterministic Finite Accepters (DFA)

Transition Graph: To visualize and represent finite automata.

- The vertices represent states. q_1
- The edges represent transitions. $\xrightarrow{a}$
- The labels on the vertices are the names of the states,
- The labels on the edges are the current values of the input symbol.
- The initial state is identified by an incoming unlabeled arrow not originating at any vertex. $\rightarrow q_0$
- The final states are drawn with a double circle. q_1

Deterministic Finite Accepters (DFA)

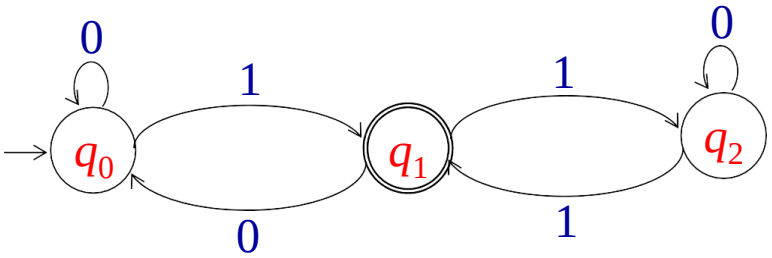
Example: Given a DFA as: $M = (Q, \Sigma, \delta, q_0, F)$ with $Q = \{q_0, q_1, q_2\}$, $\Sigma = \{0, 1\}$, $F = \{q_1\}$, δ is given by

$\delta(q_0, 0) = q_0$, (1) $\quad$ $\delta(q_0, 1) = q_1$, (4)

$\delta(q_1, 0) = q_0$, (2) $\quad$ $\delta(q_1, 1) = q_2$, (5)

$\delta(q_2, 0) = q_2$, (3) $\quad$ $\delta(q_2, 1) = q_1$, (6)

- Transition Graph:



Deterministic Finite Accepters (DFA)

Example: Given a DFA as: $M = (Q, \Sigma, \delta, q_0, F)$ with

$Q = \{q_0, q_1, q_2\}$, $\Sigma = \{0, 1\}$, $F = \{q_1\}$, δ is given by

$\delta(q_0, 0) = q_0$, (1) $\quad \delta(q_0, 1) = q_1$, (4)

$\delta(q_1, 0) = q_0$, (2) $\quad \delta(q_1, 1) = q_2$, (5)

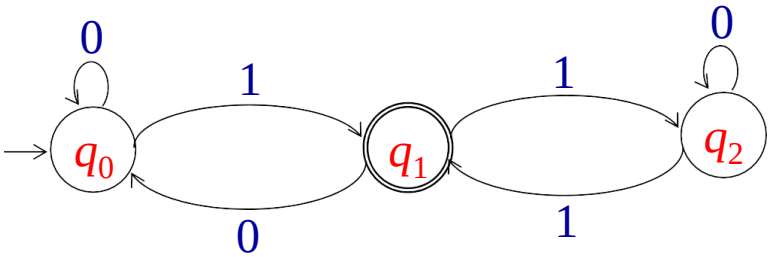
$\delta(q_2, 0) = q_2$, (3) $\quad \delta(q_2, 1) = q_1$, (6)

w =0001

- q_0, 0: $\delta(q_0, 0) = q_0$ (applied (1))
- q_0, 0: $\delta(q_0, 0) = q_0$ (applied (1))
- q_0, 0: $\delta(q_0, 0) = q_0$ (applied (1))
- q_0, 1: $\delta(q_0, 1) = q_1$ (applied (4))

As q_1 is a final state, dfa accepts w

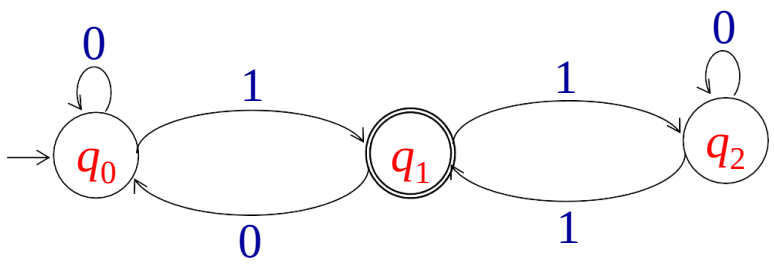
- Transition Graph:



Deterministic Finite Accepters (DFA)

Exercise

Which of the strings 100, 101, 0001, 0111, 1100, 01001, 11001, and 000011 are accepted by the dfa in the figure?



Deterministic Finite Accepters (DFA)

Extended Transition Function

$$\delta^* : Q \times \sum^* \rightarrow Q$$

- The 2nd argument of δ^* is a string, rather than a single symbol.
- δ^*'s value gives the state the automaton will be in after reading that string.
- Example
  - If $\delta(q_0, a) = q_1$, $\delta(q_1, b) = q_2$, then $\delta^*(q_0, ab) = q_2$

Deterministic Finite Accepters (DFA)

δ^* is defined recursively by:

- $\delta^*(q, \lambda) = q$,
- $\delta^*(q, wa) = \delta(\delta^*(q, w), a) = \delta(\delta^*(q, vb), a)$
  $= \delta(\delta(\delta^*(q, v), b), a) \ldots\ldots$
  $= \delta(\ldots(\delta(\delta^*(q, \lambda))\ldots), b), a)$, $\forall q \in Q, w \in \Sigma^*, a \in \Sigma$.

Deterministic Finite Accepters (DFA)

Associated Language of DFA's

Definition 2.2: The language L accepted by a dfa $M = (Q, \Sigma, \delta, q_0, F)$ is the set of all strings on Σ accepted by M. In formal notation:

$$L(M) = \{w \in \Sigma^*: \delta^*(q_0, w) \in F\}.$$

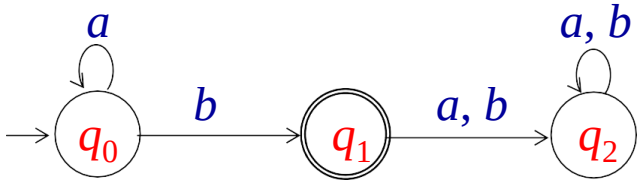
Nonacceptance means that the dfa stops in a nonfinal state, so that

$$\overline{L(M)} = \{w \in \Sigma^* : \delta^*(q_0, w) \notin F\}.$$

Deterministic Finite Accepters (DFA)

Associated Language of DFA's

Example: Consider the dfa (see fig.):



- The language L accepted by the automaton is

$$L(M) = \{a^n b : n \geq 0\}$$

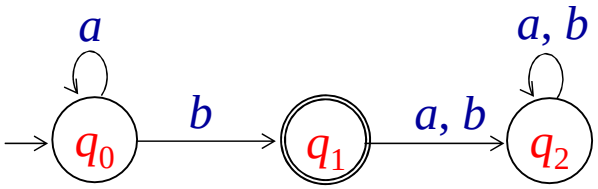
- Trap state: when the dfa goes into trap state from which it can never escape.
  - E.g., q_2 is the trap state.
  - Trap state can be the final state or not.
  - Can be defined for the trap state group which can be final or not.

Deterministic Finite Accepters (DFA)

Transition Table

- We can represent the function δ as a table:
  - The row label is the current state,
  - The column label represents the current input symbol.
  - The entry in the table defines the next state.

	a	b
q_0	q_0	q_1
q_1	q_2	q_2
q_2	q_2	q_2

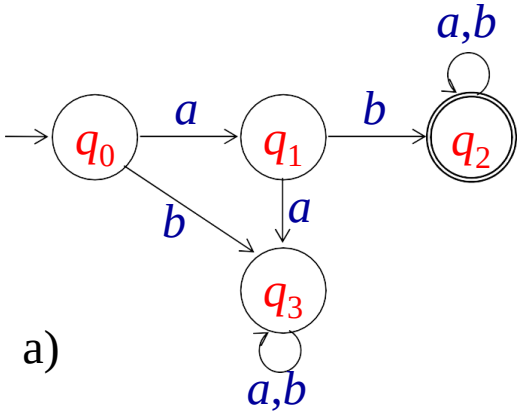


Deterministic Finite Accepters (DFA)

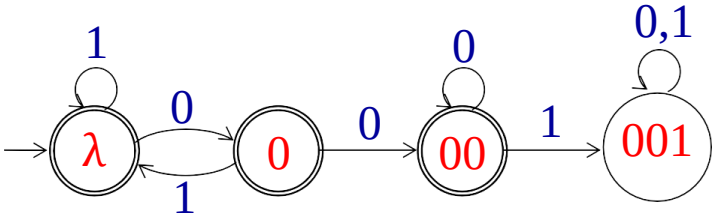
Examples:

a) Find a dfa that recognizes the set of all strings on $\Sigma = \{a,b\}$ starting with the prefix ab. (e.g, *abaaab*, *ab*, *aba*, *abb*, …)

b) Find a dfa that accepts all the strings on {0,1}, except those containing the substring 001.



a)



b)

Deterministic Finite Accepters (DFA)

Exercise

For $\Sigma = \{a,b\}$, construct dfa's that accept the sets consisting of:

(a) all strings with exactly one *a*,

(b) all strings with at least one *a*,

(c) all strings with no more than three *a*'s,

(d) all strings with at least one *a* and exactly two *b*'s.

Deterministic Finite Accepters (DFA)

Exercise

Give dfa's for the languages:

(a) $L = \{ab^5 \mathrm{w} \mathrm{b}^2 : w \in \{a,b\}^*\}$,

(b) $L = \{ab^n a^m : n \geq 2, m \geq 3\}$,

(c) $L = \{w_1 abw_2 : w_1 \in \{a,b\}^*, w_2 \in \{a,b\}^*\}$,
e.g., $w1 = \{ \lambda, a, b, aa, bb, ab, abba, \ldots\}$

(d) $\mathrm{L} = \{ba^n : n \geq 1\}$,

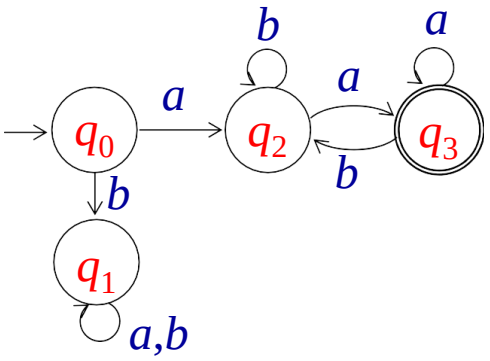
Deterministic Finite Accepters (DFA)

Regular Languages

Definition 2.3: A language L is called regular if and only if there exists some deterministic finite accepter (dfa) M such that $L = L(M)$.

E.g.: Show that the language L is regular, given $L=\{awa:w\in\{a,b\}^*\}$.

$w = \{\lambda, a, b, aaa, bbbb, abba, \ldots\}$



Deterministic Finite Accepters (DFA)

Exercise

Show that the language L= $\{a^2wb^2 : w \in \{a,b\}^*\}$ is regular.

Deterministic Finite Accepters (DFA)

Coding Exercise (Advanced)

- Input: Given a string *s* and a dfa.
- Output: Check if s is accepted by a DFA.

Deterministic Finite Accepters (DFA)

```
int m, n;

cout << "Enter numbers of rows and columns of Transition
Table: " << endl;
cin >> m >> n;

int** arTrans = new int* [n];
for (int i = 0; i < n; i++)
    arTrans[i] = new int[m];

// input array elements
for (int i = 0; i < n; ++i)
for(int j=0; j<m; j++)
{
    cout << "ArTrans[" << i << "][" << j << "] = ";
    cin >> arTrans[i][j];
}
```

Deterministic Finite Accepters (DFA)

```
//  print array elements
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < m; j++)
        cout << arTrans[i][j] << " ";
    cout << endl;
}

string s;
cout << "Input string s: ";
cin >> s;

int *arEdgeVal;
int sizeE;
// Input sizeE and arEdgeVal
 ……
```

Deterministic Finite Accepters (DFA)

Pseudocode of the function checking the validity of an input string s:

```
bool validS(int q0, string s, int arFinalState[], int sizeF,
int **arTrans, int arEdgeVal[], int sizeE){
    bool res = false;
    int len = s.size(), qt = q0;
    for (int i = 0; i < len; i++){ //traverse S
        for (int j = 0; j < sizeE; j++) //find s[i]
            if (arEdgeVal[j] == s.at(i)) break;
        int next_vertex = arTrans[qt][j];
        qt = next_vertex;
    }
    int i;
    //check if qt in arFinalState
    for(i = 0; i < sizeF; i++)
        if(qt == arFinalState[i]) break;
    if (i < sizeF) return true;
    return res;
}
```

Nondeterministic Finite Accepters (NFA)

Definition 2.4: A nondeterministic finite accepter (nfa) is defined by the quintuple:

$$M = (Q, \Sigma, \delta, q_0, F),$$

where Q, Σ, q_0, F are defined as for deterministic finite accepters, but

$$\delta : Q \times (\Sigma \cup \{\lambda\}) \rightarrow 2^Q$$

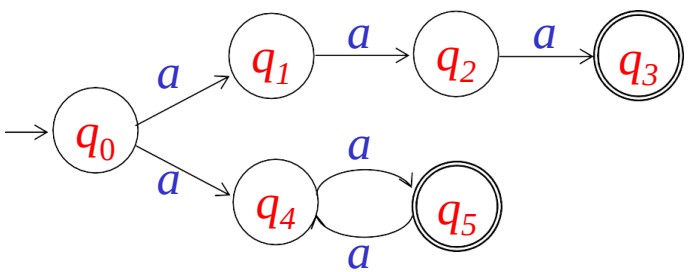
Nondeterministic Finite Accepters (NFA)

$$\delta : Q \times (\Sigma \cup \{\lambda\}) \rightarrow 2^Q$$

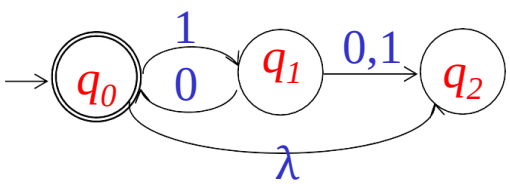
- The range of δ is in the power set 2^Q, so that its value is <u>not a single</u> element of Q but a subset of Q. This subset defines the set of all possible states that can be reached by the transition.
  - E.g., if the current state is q_1, the symbol a is read, and $\delta(q_1,a) = \{q_0, q_2\}$: then either q_0 or q_2 could be the next state of the nfa.
- λ is allowed to be the second argument of δ.
  - This means that the nfa can make a transition without consuming an input symbol.
  - Although we still assume that the input mechanism can only travel to the right, it is possible that it is stationary on some moves.
- The set $\delta(q_i,a)$ may be empty $\emptyset$, meaning that there is no transition defined for this specific situation. Such a situation is called a dead configuration, and *the automaton simply stops without further action.*

Nondeterministic Finite Accepters (NFA)

- NFA can be represented by a transition graph.
- Examples:



(a)



(b)

Nondeterministic Finite Accepters (NFA)

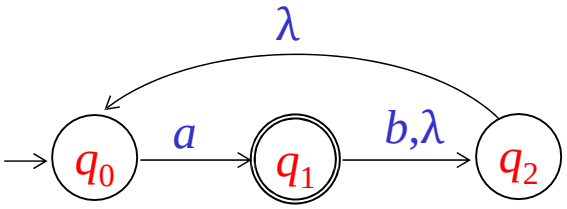
Exercise:

- Compare DFA vs NFA

Nondeterministic Finite Accepters (NFA)

Extended Transition Function

- For an nfa, the extended transition function is defined so that $\delta^*(q_i,w)$ contains q_j if and only if there is a walk/path in the transition graph from q_i to q_j labeled w. This holds for all $q_i,q_j \in Q$, and $w \in \Sigma^*$.
- Example:
  - $\delta^*(q_1, \lambda) = \{q_1, q_2, q_0\}$
  - $\delta^*(q_2, \lambda) = \{q_2, q_0\}$
  - $\delta^*(q_0, a) = \{q_1, q_2, q_0\}$
  - $\delta^*(q_1, a) = \{q_1, q_2, q_0\}$
  - $\delta^*(q_1, b) = \{q_2, q_0\}$
  - $\delta^*(q_0, b) = \emptyset$
  - $\delta^*(q_0, ab) = \{q_2, q_0\}$



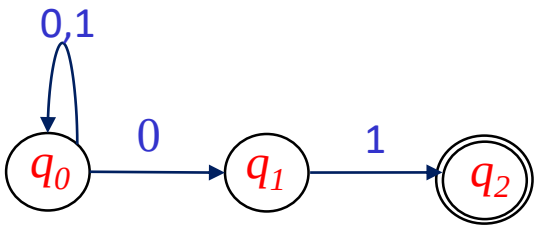
Nondeterministic Finite Accepters (NFA)

<u>Case 1</u>: How to compute $\delta^*(q_0,w)$ <u>without λ-transitions</u>.

- Compute the transition table
- Apply recursive method
  - Basis: $\delta^*(q, \lambda) = q$,
  - Induction: $\delta^*(q, wa) = \delta(\delta^*(q, w), a)$, $\forall q \in Q$, $w \in \Sigma^*$, $a \in \Sigma$

Nondeterministic Finite Accepters (NFA)

Example: Compute $\delta^*(q_0,00101)$.



- $\delta^*(q_0, \lambda) = q_0$
- $\delta^*(q_0,0) = \delta(q_0,0) = \{q_0, q_1\}$
- $\delta^*(q_0,00) = \delta(q_0,0) \cup \delta(q_1,0) = \{q_0,q_1\}$
- $\delta^*(q_0,001) = \delta(q_0,1) \cup \delta(q_1,1) = \{q_0\} \cup \{q_2\} = \{q_0,q_2\}$
- $\delta^*(q_0,0010) = \delta(q_0,0) \cup \delta(q_2,0) = \{q_0,q_1\} \cup \varphi = \{q_0,q_1\}$
- $\delta^*(q_0,00101) = \delta(q_0,1) \cup \delta(q_1,1) = \{q_0\} \cup \{q_2\} = \{q_0,q_2\}$

Is "00101" accepted by the above NFA? → slide 35

Nondeterministic Finite Accepters (NFA)

<u>Case 2</u>: How to compute $\delta^*(q_0, w)$ <u>with λ-transitions</u>.

- Compute the transition table
- Apply recursive method
  - Basis: $\delta^*(q, \lambda) = ?$,
  - Induction: $\delta^*(q, wa) = \delta(\delta^*(q, w), a)$, $\forall q \in Q$, $w \in \Sigma^*$, $a \in \Sigma$.
  - $\delta^*(q, \mathrm{a}) = ?$

Nondeterministic Finite Accepters (NFA)

Let T be a subset of Q

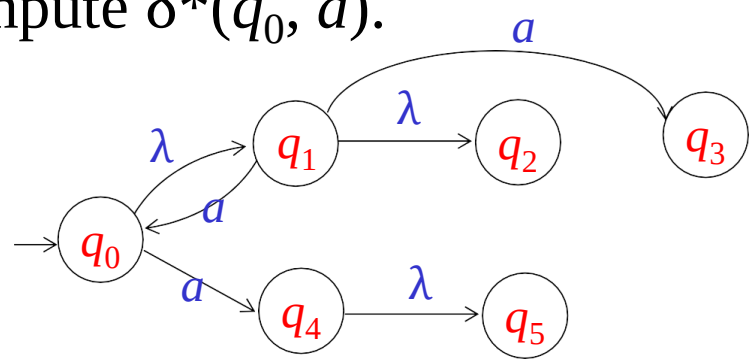
$\delta(T, a) = \cup_{q \in T} \delta(q, a)$; $\delta^*(T, \lambda) = \cup_{q \in T} \delta(q, \lambda)$;

Let $move(q, a)$, $move(T, a)$, $\lambda\text{-}closure(q)$, $\lambda\text{-}closure(T)$ be $\delta(q, a)$, $\delta(T, a)$, $\delta^*(q, \lambda)$, $\delta^*(T, \lambda)$.

- $\delta^*(q, a) = \lambda\text{-}closure(move(\lambda\text{-}closure(q), a))$
- $\delta^*(T, a) = \lambda\text{-}closure(move(\lambda\text{-}closure(T), a))$

Nondeterministic Finite Accepters (NFA)

Example: Compute $\delta^*(q_0, a)$.



	a	λ
q_0	q_4	q_1
q_1	q_0, q_3	q_2
q_2		
q_3		
q_4		q_5
q_5		

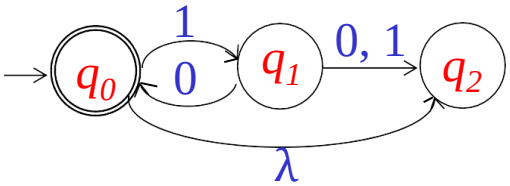
- $\delta^*(q_0, a) = \lambda\text{-}closure(move(\lambda\text{-}closure(q_0), a))$
- $\lambda\text{-}closure(q_0) = \{q_0, q_1, q_2\}$
- $move(\{q_0, q_1, q_2\}, a) = \{q_4, q_0, q_3\}$
- $\lambda\text{-}closure(\{q_4, q_0, q_3\}) = \{q_4, q_0, q_3, q_5, q_1, q_2\}$

So, $\delta^*(q_0, a) = \{q_0, q_1, q_2, q_3, q_4, q_5\}$

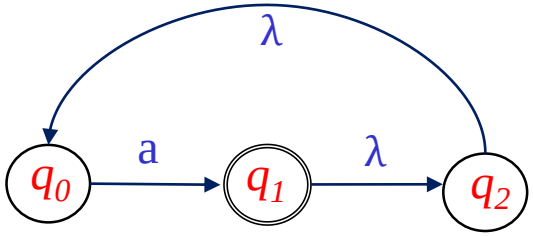
Nondeterministic Finite Accepters (NFA)

Exercise:

1. Find a) $\delta^*(q_0, 1011)$ and b) $\delta^*(q_1, 01)$ in the below figure.



2. Find a) $\delta^*(q_0, a)$ and b) $\delta^*(q_1, \lambda)$ in the below figure.



Nondeterministic Finite Accepters (NFA)

Definition 2.6: The language L accepted by an nfa $M = (Q, \Sigma, \delta, q_0, F)$ is defined as the set of all strings accepted in the above sense. Formally,

$$L(M) = \{w \in \Sigma^*: \delta^*(q_0, w) \cap F \neq \emptyset\}.$$

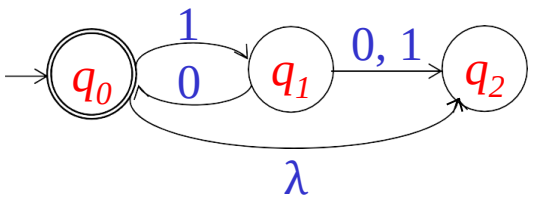
In words, the language consists of all strings w for which there is a walk labeled w from the initial vertex of the transition graph to some final vertex.

Example: consider the example on the slide 30

Nondeterministic Finite Accepters (NFA)

Example: Language L is accepted by the below NFA:

$$L = \{(10)^n : n \geq 0\}$$



Nondeterministic Finite Accepters (NFA)

Exercise

1. Construct an nfa with three states that accepts the language $\{ab, abc\}^*$
2. Find an nfa with three states that accepts the language

$$L = \{a^n : n \geq 1\} \cup \{b^m a^k : m \geq 0, k \geq 0\}$$

3. Find an nfa with four states for $L = \{a^n : n \geq 0\} \cup \{b^n a : n \geq 1\}$.

Nondeterministic Finite Accepters (NFA)

Coding Exercise (Advanced)

- Input an NFA by a transition table.
- Input q_0 and an edge label a.
- Compute and output the results of
  - $\delta^*(q, a) = \lambda\text{-}closure(move(\lambda\text{-}closure(q), a))$
  - $\delta^*(T, a) = \lambda\text{-}closure(move(\lambda\text{-}closure(T), a))$

Nondeterministic Finite Accepters (NFA)

PseudoCode

```
// define the empty entry in the transition table arTrans is -1
void find_lamda_closure(int q, int N /*row size*/, int M /*col size*/,
int** arTrans, int& nLC, int* arLC)
{
    arLC = new int[N];

    nLC = 0; //size of arLC
    //M-1 is the index of the last column
    while(arTrans[q][M-1] >= 0)
    {
        q = arTrans[q][M - 1];
        arLC[nLC] = q;
        nLC++;
    }

    arLC = (int*)realloc(arLC, nLC * sizeof(int));
}
```

Nondeterministic Finite Accepters (NFA)

```
//int* arMoveA contains the states received from a transition a from q
void move(int q, int a, int N /*row size*/, int M /*col size*/, int**
arTrans, int sizeE, int* arEdgeVal, int*arMoveA)
{
    //find the column index associated with a
    int j;
    for (j = 0; j < sizeE; j++) //find j of a
        if (arEdgeVal[j] == a) break;

    //assume that the empty value is -1 in the transition table
    int i = 0;
    while (arTrans[q][j] != -1)
    {
        arMoveA[i] = arTrans[q][j];
        i++;
        q = arTrans[q][j];
    }
}
```

Solution of the case each entry is a combination of more than one states?

Nondeterministic Finite Accepters (NFA)

```
//define the T-closure
void find_T_closure(int* T, int nT /*size of T*/, int N /*row size*/,
int M /*col size*/, int** arTrans, int& nTC, int* arTC)
{
    nTC = 0;
    arTC = new int[N];

    int nLC;
    int** arLC;

    //allocate memory to arLC

    //find lamda-closure for each qi in T
    for (int i = 0; i < nTC; i++) //find lamda-closure for each qi in T
```

Nondeterministic Finite Accepters (NFA)

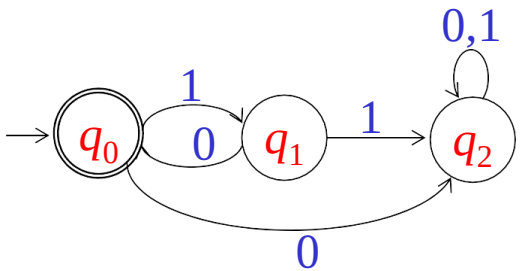
```
    {
        find_lamda_closure(T[i], N, M, arTrans, nLC, arLC[i]);
        int count = 0;//count of new elements to be added into arTC
        for (int j = 0; j < nTC; j++) // check if any element in NLC
exists in arTC? If not, add it into arTC
        {
            for (int k = 0; k < nLC; k++)
                if (arLC[i][k] == arTC[j]) break;
            if (k == nLC)
            {
                arTC[nTC + count] = arLC[i][k];
                count ++;
            }
        }
        nTC += count;
    }
}
```

Equivalence between DFA vs NFA

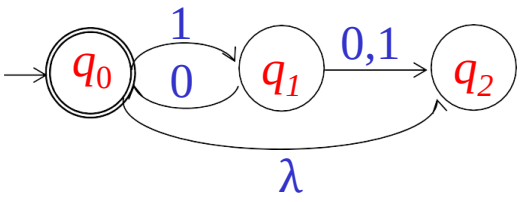
Definition 2.7: Two finite accepters, M_1 and M_2, are said to be equivalent if they both accept the same language.

$$L(M_1) = L(M_2)$$

Example: The dfa and nfa are equivalent since they both accept the language $L = \{(10)^n : n \geq 0\}$.



dfa



nfa

Equivalence between DFA vs NFA

Discussion

- A dfa is a restricted kind of nfa, it is clear that any language that is accepted by a dfa is also accepted by some nfa.
- There is a language accepted by some nfa for which, in principle, we cannot find a dfa. But it turns out that this is not so.
- The classes of dfa's and nfa's are equally powerful: For every language accepted by some nfa there is a dfa that accepts the same language.

Equivalence between DFA vs NFA

- Theorem 2.2: Let L be the language accepted by a nondeterministic finite accepter (NFA) $M_N = (Q_N,\Sigma,\delta_N,q_0,F_N)$. Then there exists a deterministic finite accepter (DFA) $M_D = (Q_D,\Sigma,\delta_D,q_0,F_D)$ such that

$$L = L(M_D)$$

- Procedure nfa-to-dfa
  - Input: nfa $M_N = (Q_N, \Sigma, \delta_N, q_0, F_N)$
  - Output: Transition graph G_D of associated dfa M_D

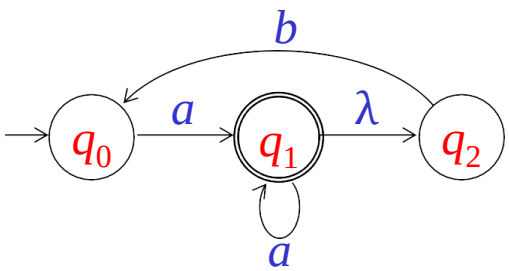
Equivalence between DFA vs NFA

1. Create a graph G_D with vertex $\delta_N^*(q_0, \lambda)$. Identify this vertex as the initial vertex.
2. Repeat the following steps 3-6 until no more edges are missing.
3. Take any vertex $\{q_i, q_j, \ldots, q_k\}$ of G_D that has no outgoing edge for some $a \in \Sigma$.
4. Compute $\delta_N^*(\{q_i, q_j, \ldots, q_k\}, a) = \delta_N^*(q_i, a) \cup \delta_N^*(q_j, a) \cup \ldots \cup \delta_N^*(q_k, a) = \{q_l, q_m, \ldots, q_n\}$.
5. Create a vertex for G_D labeled $\{q_l, q_m, \ldots, q_n\}$ if it does not already exist.
6. Add to G_D an edge from $\{q_i, q_j, \ldots, q_k\}$ to $\{q_l, q_m, \ldots, q_n\}$ and label it with a.
7. Every state of G_D whose label contains any $q_f \in F_N$ is identified as a final vertex.

Equivalence between DFA vs NFA

Example

- Convert the nfa to an equivalent dfa.



	a	b	λ
q_0	q_1		
q_1	q_1		q_2
q_2		q_0	

$\delta^*(q_0, \lambda) = \{\boldsymbol{q_0}\}$

$\delta^*(\{\boldsymbol{q_0}\}, a) = \{\boldsymbol{q_1}, \boldsymbol{q_2}\}$

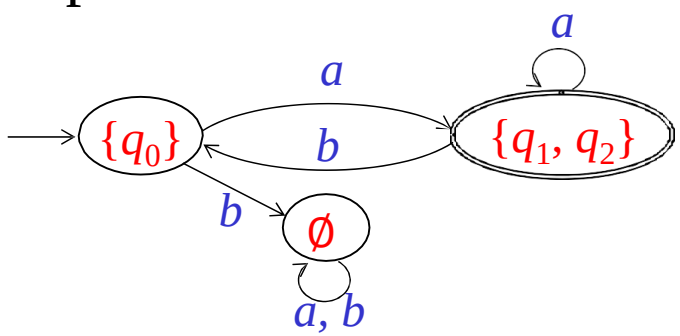
$\delta^*(\{\boldsymbol{q_0}\}, b) = \boldsymbol{\emptyset}$

$\delta^*(\{\boldsymbol{q_1}, \boldsymbol{q_2}\}, a) = \{\boldsymbol{q_1}, \boldsymbol{q_2}\}$

$\delta^*(\{\boldsymbol{q_1}, \boldsymbol{q_2}\}, b) = \{\boldsymbol{q_0}\}$

Equivalence between DFA vs NFA

Output dfa



$\delta^*(q_0, \lambda) = \{\boldsymbol{q_0}\}$

$\delta^*(\{\boldsymbol{q_0}\}, a) = \{\boldsymbol{q_1}, \boldsymbol{q_2}\}$

$\delta^*(\{\boldsymbol{q_0}\}, b) = \boldsymbol{\emptyset}$

$\delta^*(\{\boldsymbol{q_1}, \boldsymbol{q_2}\}, a) = \{\boldsymbol{q_1}, \boldsymbol{q_2}\}$

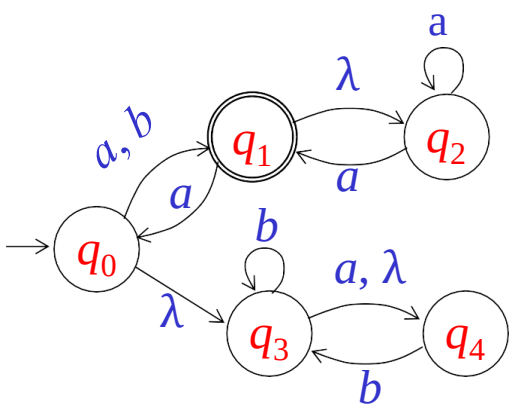
$\delta^*(\{\boldsymbol{q_1}, \boldsymbol{q_2}\}, b) = \{\boldsymbol{q_0}\}$

	a	b	λ
q_0	q_1		
q_1	q_1		q_2
q_2		q_0	

- A state of nfa is a set of states of dfa.
- A final state of dfa contains a final state of nfa.

Equivalence between DFA vs NFA

- Find an associated dfa from the below nfa:



	a	b	λ
q_0	q_1	q_1	q_3
q_1	q_0		q_2
q_2	q_1, q_2		
q_3	q_4	q_3	q_4
q_4		q_3	

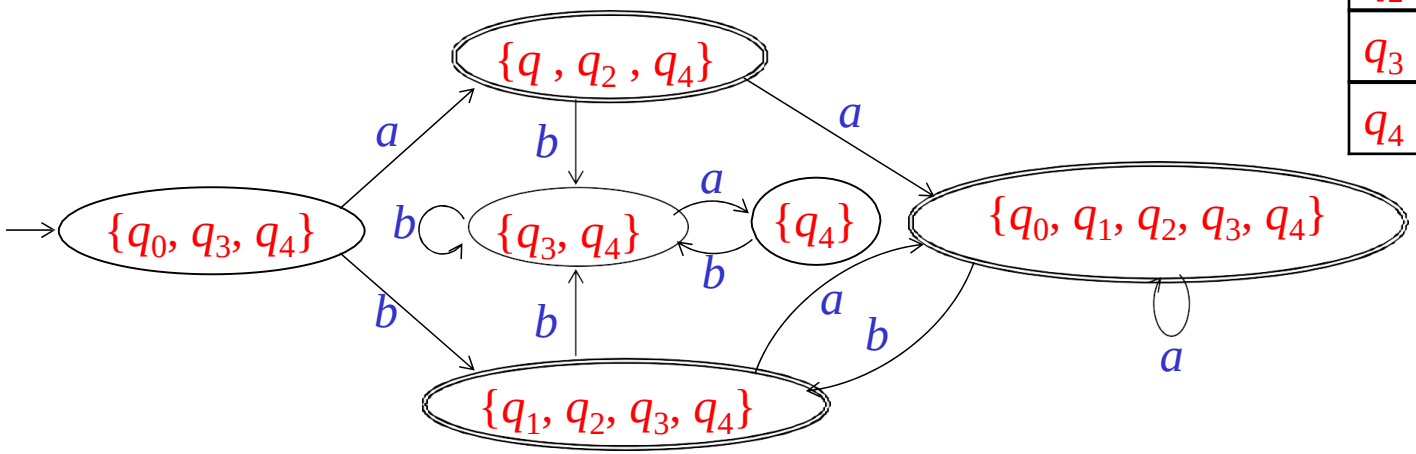
Equivalence between DFA vs NFA

- $\delta_N^*(q_0, \lambda) = \mathbf{\{q_0, q_3, q_4\}}$
- $\delta_N^*(\{q_0, q_3, q_4\}, a) = \mathbf{\{q_1, q_2, q_4\}}$
- $\delta_N^*(\{q_0, q_3, q_4\}, b) = \mathbf{\{q_1, q_2, q_3, q_4\}}$
- $\delta_N^*(\{q_1, q_2, q_4\}, a) = \mathbf{\{q_0, q_1, q_2, q_3, q_4\}}$
- $\delta_N^*(\{q_1, q_2, q_4\}, b) = \mathbf{\{q_3, q_4\}}$
- $\delta_N^*(\{q_1, q_2, q_3, q_4\}, a) = \{q_0, q_1, q_2, q_3, q_4\}$
- $\delta_N^*(\{q_1, q_2, q_3, q_4\}, b) = \{q_3, q_4\}$
- $\delta_N^*(\{q_0, q_1, q_2, q_3, q_4\}, a) = \{q_0, q_1, q_2, q_3, q_4\}$
- $\delta_N^*(\{q_0, q_1, q_2, q_3, q_4\}, b) = \{q_1, q_2, q_3, q_4\}$
- $\delta_N^*(\{q_3, q_4\}, a) = \mathbf{\{q_4\}}$ $\quad$ $\delta_N^*(\{q_3, q_4\}, b) = \{q_3, q_4\}$
- $\delta_N^*(\{q_4\}, a) = \emptyset$ $\quad$ $\delta_N^*(\{q_4\}, b) = \{q_3, q_4\}$

	a	b	λ
q_0	q_1	q_1	q_3
q_1	q_0		q_2
q_2	q_1, q_2		
q_3	q_4	q_3	q_4
q_4		q_3	

Equivalence between DFA vs NFA

	a	b	λ
q_0	q_1	q_1	q_3
q_1	q_0		q_2
q_2	q_1, q_2		
q_3	q_4	q_3	q_4
q_4		q_3	



Equivalence between DFA vs NFA

- Find the associated dfa of the following nfa:

Nfa M_1			
	a	b	λ
q_0	q_1	q_3	q_1
q_1	q_2	q_2, q_0	
q_2		q_1	
q_3	q_0, q_4	q_3	q_4
q_4	q_3, q_4	q_4	
$F = \{q_2\}$			

Nfa M_2			
	a	b	λ
q_0	q_1, q_3	q_3	q_3
q_1	q_2	q_2	q_0
q_2	q_1		
q_3	q_4		q_4
q_4	q_4	q_3	
$F = \{q_4\}$			

Nfa M_3			
	a	b	λ
q_0	q_1	q_2	q_1
q_1	q_1, q_2	q_3	q_3
q_2		q_0, q_2	
q_3	q_2, q_3		
$F = \{q_0\}$			

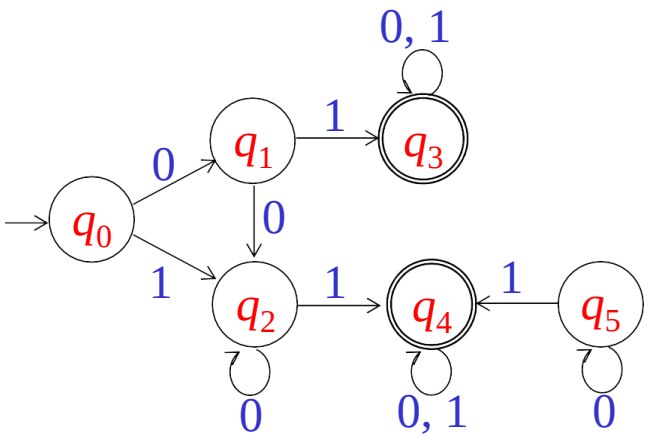
Equivalence between DFA vs NFA

Coding exercise (Advanced)

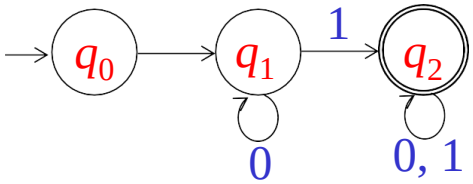
- Input an NFA.
- Output an equivalent DFA using the algorithm nfa2dfa on page 46.

Reduction of the Number of States in Finite Automata

- Given two equivalent dfas below:



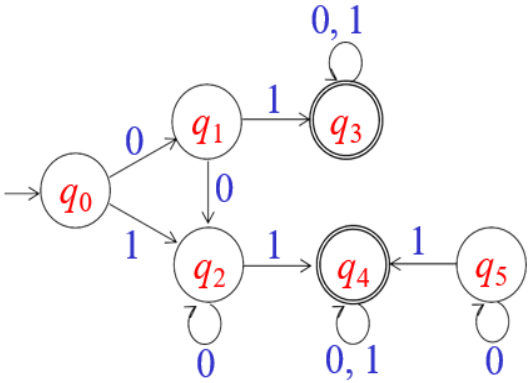
(*a*)



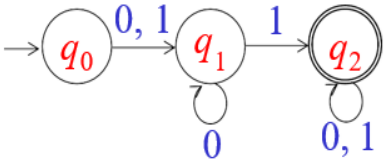
(*b*)

Reduction of the Number of States in Finite Automata

- Inaccessible state is the state that can never be reached from the initial state.
  - E.g., the state q_5 plays absolutely no role in the automaton since it can never be reached from the initial state q_0.
- An inaccessible state can be removed (along with all transitions relating to it) without affecting the language accepted by the automaton.



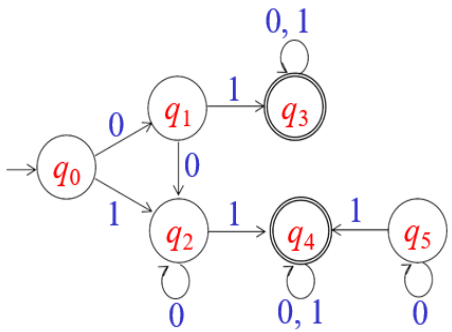
(a)



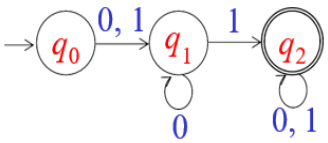
(b)

Reduction of the Number of States in Finite Automata

- In figure a), the states reachable subsequent to the first move $\delta(q_0, 0)$ mirror those reachable from a first move $\delta(q_0, 1)$. The second automaton combines these two options.
- The second alternative (Figure b) is clearly preferable. Representation of an automaton for the purpose of computation requires space proportional to the number of states. For storage efficiency, it is desirable to reduce the number of states as far as possible.



(a)



(b)

Reduction of the Number of States in Finite Automata

Definition 2.8:

Two states p and q of a dfa are called indistinguishable if

$$\delta^*(q, w) \in F \text{ implies } \delta^*(p, w) \in F,$$

$$\text{and } \delta^*(q, w) \notin F \text{ implies } \delta^*(p, w) \notin F,$$

for all $w \in \Sigma^*$.

If, on the other hand, there exists some string $w \in \Sigma^*$ such that

$$\delta^*(q, w) \in F \text{ and } \delta^*(p, w) \notin F,$$

then the states p and q are said to be distinguishable by a string w.

Reduction of the Number of States in Finite Automata

- Two states are either indistinguishable or distinguishable.
- In-distinguishability has the properties of an equivalence relation: If p and q are indistinguishable and if q and r are also indistinguishable, then so are p and r, and all three states are indistinguishable.
- One method for reducing the states of a dfa is based on finding and combining indistinguishable states.

Reduction of the Number of States in Finite Automata

- A method for finding pairs of <u>distinguishable</u> states.

procedure: mark()

Input: ($|Q| \times (|Q| - 1)/2$) pairs of (p, q) of a full dfa

Output: pairs of (p, q) are marks

1. Remove all inaccessible states.
   - This can be done by enumerating all simple paths of the graph of the dfa starting at the initial state.
   - Any state not part of some path is inaccessible.

Reduction of the Number of States in Finite Automata

2. Consider all pairs of states (p, q). If $p \in F$ and $q \notin F$ or vice versa, mark the pair (p, q) as distinguishable.

3. Repeat the following step until no previously unmarked pairs are marked.

   i. For all pairs (p, q) and all $a \in \Sigma$, compute $\delta(p, a) = p_a$ and $\delta(q, a) = q_a$.

   ii. If the pair (p_a, q_a) is marked as distinguishable, mark (p, q) as distinguishable.

Reduction of the Number of States in Finite Automata

- Theorem 2.3: The procedure *mark*, applied to any dfa $M = (Q, \lambda, \delta, q_0, F)$, terminates and determines all pairs of <u>distinguishable</u> states.

- The states q_i and q_j are distinguishable with a string of length n if and only if there are transitions:

$$\delta(q_i, a) = q_k \text{ and } \delta(q_j, a) = q_l$$

for some $a \in \Sigma$, with q_k and q_l distinguishable by a string of length n-1.

Reduction of the Number of States in Finite Automata

- The procedure mark can be implemented by partitioning the states into equivalence classes.
- Whenever two states are found to be distinguishable, they are immediately put into separate equivalence classes.

Reduction of the Number of States in Finite Automata

- Once the indistinguishability classes are found, the construction of the minimal dfa is straightforward.
- Given a dfa $M = (Q, \Sigma, \delta, q_0, F)$, we construct a reduced dfa $\widehat{M} = (\hat{Q}, \Sigma, \hat{\delta}, \widehat{q_0}, \hat{F})$

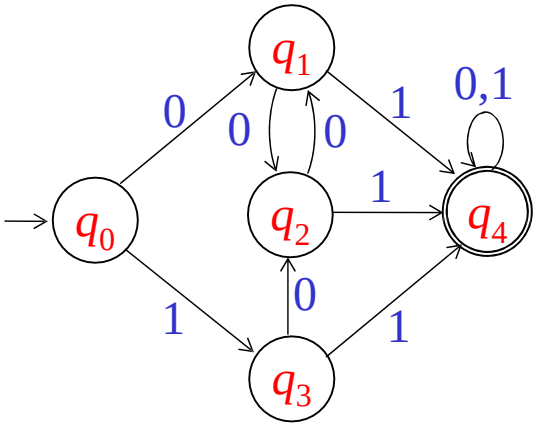
Reduction of the Number of States in Finite Automata

procedure: reduce()

1. Use procedure *mark* to generate the equivalence classes, say $\{q_i, q_j, \ldots, q_k\}$.
2. For each set $\{q_i, q_j, \ldots, q_k\}$ of such indistinguishable states, create a state labled $ij\ldots k$ for $\widehat{M}$.
3. For each transition rule of M of the form $\delta(q_r,a)=q_p$, find the sets to which q_r and q_p belong. If $q_r \in \{q_i, q_j, \ldots, q_k\}$ and $q_p \in \{q_l, q_m, \ldots, q_n\}$, add to $\hat{\delta}$ a rule $\hat{\delta}(ij\ldots k,a) = lm\ldots n$.
4. The initial state $\widehat{q_0}$ is that state of $\widehat{M}$ whose label includes the 0.
5. $\hat{F}$ is the set of all the states whose label contains i such that $q_i \in F$.

Example

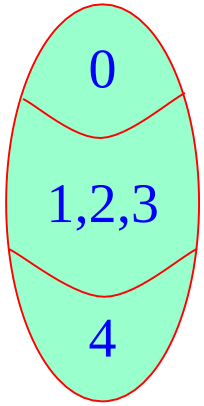
- Reduce the number of the states of the below dfa:

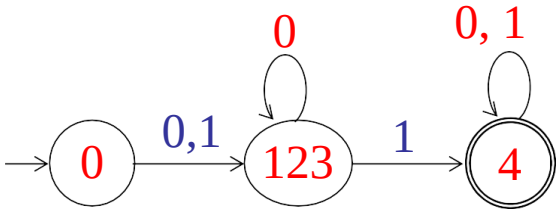


	0	1
q_0	q_1	q_3
q_1	q_2	q_4
q_2	q_1	q_4
q_3	q_2	q_4
$\boldsymbol{q_4}$	q_4	q_4

(q_0,q_1)✓1	(q_0,q_3)✓1	$\boldsymbol{(q_1, q_2)}$	(q_1,q_4)✓0	(q_2,q_4)✓0
(q_0,q_2)✓1	(q_0,q_4)✓0	$\boldsymbol{(q_1, q_3)}$	$\boldsymbol{(q_2,q_3)}$	(q_3,q_4)✓0

	0	1
q_0	q_1	q_3
q_1	q_2	q_4
q_2	q_1	q_4
q_3	q_2	q_4
$\boldsymbol{q_4}$	q_4	q_4





Reduction of the Number of States in Finite Automata

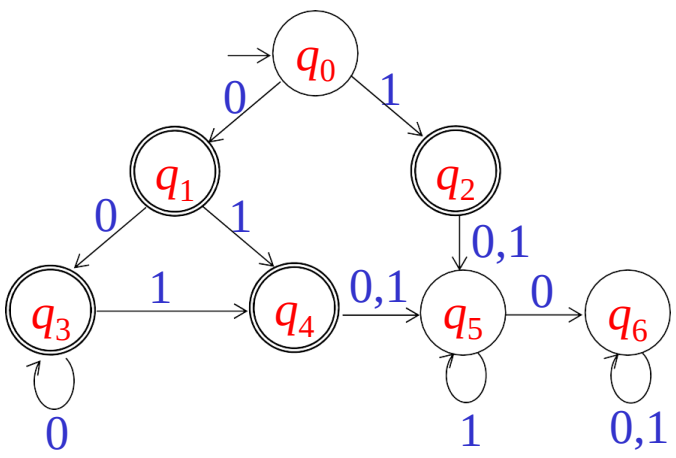
Theorem 2.4: Given any dfa M, application of the procedure reduce yields another dfa $\widehat{M}$ such that

$$L(M) = L(\widehat{M})$$

Futhermore, $\widehat{M}$ is minimal in the sense that there is no other dfa with a smaller number of states that also accepts $L(M)$.

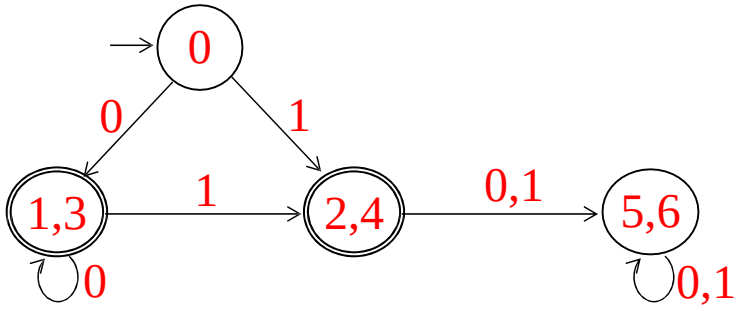
Reduction of the Number of States in Finite Automata

Reduce the number of the states of the below dfa.



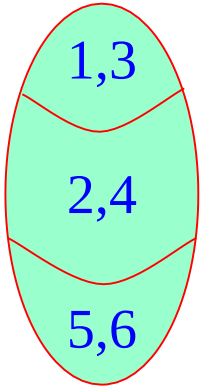
	0	1
q_0	q_1	q_2
$\boldsymbol{q_1}$	q_3	q_4
$\boldsymbol{q_2}$	q_5	q_5
$\boldsymbol{q_3}$	q_3	q_4
$\boldsymbol{q_4}$	q_5	q_5
q_5	q_6	q_5
q_6	q_6	q_6

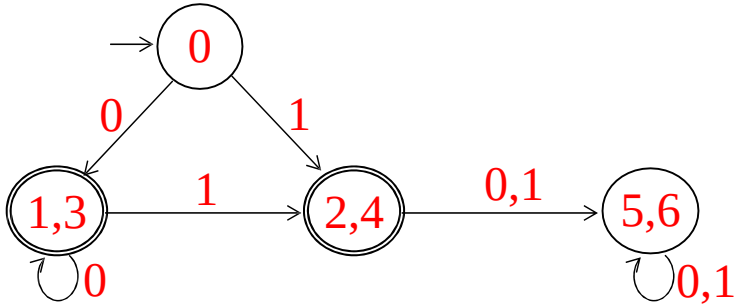
	0	1
q_0	q_1	q_2
$\boldsymbol{q_1}$	q_3	q_4
$\boldsymbol{q_2}$	q_5	q_5
$\boldsymbol{q_3}$	q_3	q_4
$\boldsymbol{q_4}$	q_5	q_5
q_5	q_6	q_5
q_6	q_6	q_6



(q_0, q_1)✓0	(q_1, q_2)✓1	$\mathbf{(q_2, q_4)}$	(q_4,q_5)✓0
(q_0, q_2)✓0	$\mathbf{(q_1, q_3)}$	(q_2, q_5)✓0	(q_4, q_6)✓0
(q_0, q_3)✓0	(q_1, q_4)✓1	(q_2, q_6)✓0	$\mathbf{(q_5, q_6)}$
(q_0, q_4)✓0	(q_1, q_5)✓0	(q_3, q_4)✓1	
(q_0, q_5)✓1	(q_1, q_6)✓0	(q_3, q_5)✓0	
(q_0, q_6)✓1	(q_2, q_3)✓1	(q_3, q_6)✓0	

	0	1
q_0	q_1	q_2
$\mathbf{q_1}$	q_3	q_4
$\mathbf{q_2}$	q_5	q_5
$\mathbf{q_3}$	q_3	q_4
$\mathbf{q_4}$	q_5	q_5
q_5	q_6	q_5
q_6	q_6	q_6





Reduction of the Number of States in Finite Automata

Dfa M_1		
	a	b
q_0	q_1	q_4
$\boldsymbol{q_1}$	q_4	q_2
q_2	q_4	q_3
q_3	q_3	q_3
$\boldsymbol{q_4}$	q_4	q_5
q_5	q_4	q_6
q_6	q_7	q_6
q_7	q_7	q_7

Dfa M_2		
	a	b
q_0	q_1	q_2
q_1	q_2	q_3
q_2	q_2	q_3
q_3	q_5	q_4
q_4	q_5	q_3
$\boldsymbol{q_5}$	q_5	q_5
q_6	q_1	q_7
q_7	q_6	q_4

Dfa M_3		
	a	b
q_0	q_1	q_2
q_1	q_2	q_3
q_2	q_1	q_4
$\boldsymbol{q_3}$	q_4	q_0
$\boldsymbol{q_4}$	q_3	q_0

Dfa M_4		
	a	b
q_0	q_1	q_3
q_1	q_2	q_4
q_2	q_0	q_3
$\boldsymbol{q_3}$	q_1	q_4
$\boldsymbol{q_4}$	q_2	q_3

Exercise: Reduce the number of the states of the below dfa.

Reduction of the Number of States in Finite Automata

Coding exercise (Advanced)

- Input a dfa and its size n from the console or a file.
- Output all equivalence classes using the algorithm *mark()* on pages 59, 60.
- Output a new dfa after reducing the states as the algorithm *reduce()* on page 64.

References

1. Peter Linz: An Introduction to Formal Languages and Automata, Fifth edition, Jones & Bartelett learning, 2012.
2. Bài giảng môn Lý thuyết Automat và Ngôn ngữ Hình Thức, Hồ Văn Quân, Đại học Kỹ Thuật TPHCM.

Deterministic Finite Accepters (DFA)

Solution

For Σ = {a,b}, construct dfa's that accept the sets consisting of

d) all strings with at least one *a* and exactly two *b*'s
(e.g., *a..abb*, *ba..ab*, *bba..a*, *a..abva..aba...a*,…)

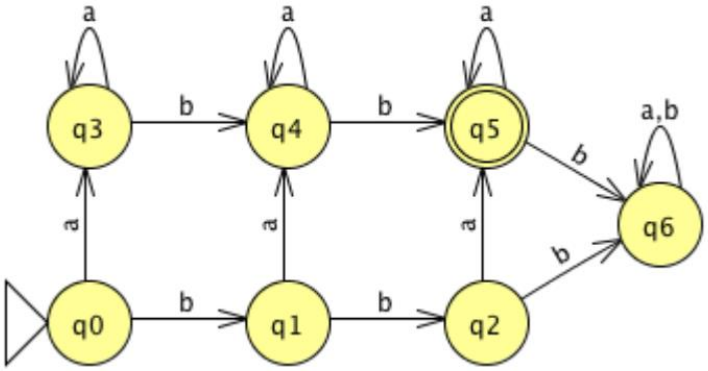
M =(Q, Σ, δ, q_0, *F*) with

Q = {q_0, q_1, q_2, q_3, q_4, q_5, q_6},

Σ = {*a*, *b*},

F = {q_5},

δ as in the transition graph beside

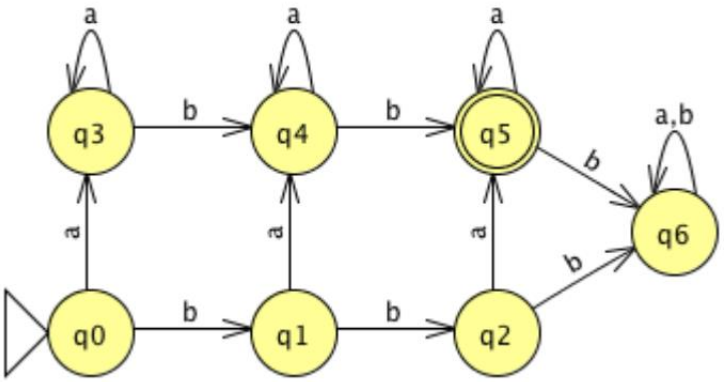


Deterministic Finite Accepters (DFA)

Exercise

For $\Sigma = \{a,b\}$, construct dfa's that accept the sets consisting of

d) all strings with at least one *a* and exactly two *b*'s $\Leftrightarrow$ $L(M) = \{a^n b^2 : n \geq 1\}$



Back

Nondeterministic Finite Accepters (NFA)

$M = (Q, \Sigma, \delta, q_0, F)$,

- DFA:

  $\delta: Q \times \Sigma \rightarrow Q$

- NFA:

  $\delta: Q \times (\Sigma \cup \{\lambda\}) \rightarrow 2^Q$

Back